B.TECH CCE REVISION · PART 8

Software Design, Product and Project Management

Covers the marksheet subject Software Design Technology, plus the management parts of Engineering Economics and Essentials of Management that apply to software.

This is what separates building a thing from running a product with a team. It was flagged as more important than the purely technical modules, and it's where an MBA and a CS degree overlap most. The chapter is written from the seat of someone who will lead software work, often while also writing some of it with AI: enough design vocabulary to read a design document and judge a trade-off, and enough process knowledge to keep a project honest.

How software gets built by teams: the lifecycle and agile, requirements and prioritisation, architecture and technical debt, design principles and patterns, estimation, testing and release, and how a non-engineer works well with engineers.

> **How to read it.** Sections 1–3 are the product side, 4–6 the design side, 7–9 the delivery side, 10 the working relationship. The main lab is writing a one-page design doc for something you're building.

Contents

1. The software development lifecycle, and why agile happened
2. Requirements: problems, users, stories, scope
3. Prioritisation and roadmaps
4. Architecture basics
5. Design principles and patterns
6. Technical debt
7. Estimation and managing scope and risk
8. Testing strategy
9. Releasing and running software
10. Working with engineers as a non-engineer
11. Building with AI coding tools
12. Labs
13. Self-check questions and answers
14. Resources

Isaqi · November 2026 · Written from general knowledge of the field as of late 2026; check tool versions, prices and vendor specifics against current documentation.

1. The lifecycle, and why agile happened

Every method does the same activities (understand, design, build, test, release, maintain); they differ in how big each loop is and how much is decided up front.

Approach	Shape	Works when	Fails when
Waterfall	Requirements → design → build → test → release, each signed off before the next	Requirements are stable and well understood; heavy regulation; fixed-scope contracts	Requirements change or users don't know what they want until they see it (most software)
Iterative / incremental	Build a small working slice, extend it in cycles	Most products	Without discipline it becomes endless rework
Scrum	Fixed sprints (1–4 weeks), a prioritised backlog, planning, daily stand-up, review, retrospective; roles of product owner, scrum master, team	Teams needing rhythm and visible progress	Ceremonies become theatre; sprint commitments are gamed; "agile" used as an excuse for no planning
Kanban	Continuous flow; visualise work on a board; limit work in progress	Support, operations, small teams with unpredictable work	No natural checkpoint to step back and re-plan
Shape Up (Basecamp)	Shape work up front, bet on 6-week cycles, fixed time and variable scope, cool-down between	Small senior teams	Needs strong shaping skill and authority to cut scope

The **Agile Manifesto** (2001) valued working software over documentation, collaboration over contracts, and responding to change over following a plan. The core insight is economic: **feedback is the scarcest input**, so shorten the time between deciding something and learning whether it was right. Everything else (sprints, stand-ups, story points) is optional machinery in service of that.

Common failure modes regardless of method: no clear owner of priorities; too much work in progress (everything 80% done, nothing shipped); the team never talks to users; and "done" meaning "code written" rather than "in users' hands and working".

2. Requirements

Start from the problem and the user, not the feature. Most failed software solved the wrong problem well.

- **Problem statement first**: who has the problem, how often, how painful, what they do today, and how you'd know it's solved. "Tier-2 college students fail interviews because they freeze in English" is a problem; "AI mock interviews" is a solution.
- **Jobs to be done**: what is the user trying to get done ("feel ready for Thursday's placement interview")? Features are hired for that job.
- **User stories**: *As a [user], I want [capability] so that [outcome].* "As a student, I want to see which filler words I overuse so that I can practise cutting them." Small, testable, valuable on its own.
- **Acceptance criteria**: concrete conditions for "done", often as *Given / When / Then*. "Given a 2-minute answer, when scoring finishes, then the student sees the three most frequent filler words with counts, within 60 seconds."
- **Non-functional requirements**: performance, availability, security, privacy, accessibility, cost per user, works on low-end Android over 4G. Often decisive and often forgotten.
- **Scope and non-goals**: write down what you're *not* doing in this version. Non-goals prevent scope creep better than goals do.
- **MVP**: the smallest thing that tests the riskiest assumption with real users. Not "version 1 with fewer features", and not an excuse for something that doesn't work. Sometimes the MVP is a WhatsApp group and a human coach, with no code at all.

3. Prioritisation and roadmaps

Saying no to good ideas is the job. Frameworks help make the trade-offs visible; they don't make them for you.

Method	How it works	Use it for
Impact vs effort	2×2 grid; do high-impact, low-effort first	Quick triage
RICE	(Reach × Impact × Confidence) ÷ Effort	Comparing many candidate features with some data
MoSCoW	Must / Should / Could / Won't (this time)	Scoping a release
Kano	Basic expectations vs performance features vs delighters	Understanding which features users notice by absence
Cost of delay	How much value is lost per week of not having it	Sequencing; urgent vs merely important

Roadmaps: a **Now / Next / Later** roadmap communicates direction without pretending to know dates for things months away. Tie items to outcomes ("increase week-2 retention") rather than features where you can, so the team can find better solutions than the first idea.

Metrics: pick a **north-star metric** that captures delivered value (e.g. weekly practice sessions completed, not sign-ups), supported by input metrics the team can move. Beware vanity metrics that only go up.

4. Architecture basics

Architecture is the set of decisions that are expensive to change later. Make few of them early, and make them reversible where you can.

Monolith vs microservices

	Modular monolith	Microservices
Shape	One deployable app, organised into clear internal modules	Many small services, each deployed independently, talking over the network
Strengths	Simple to build, test, deploy and debug; one database; fast for small teams	Independent scaling and deployment; teams own services end to end; failure isolation
Costs	Can become tangled; one deploy for everything	Network calls (Part 7's problems everywhere), distributed data, many pipelines, operational overhead
Right for	Almost every startup and most products for years	Large organisations with many teams that block each other

Start with a well-structured monolith. Microservices solve an organisational scaling problem (many teams) more than a technical one. Splitting too early buys the costs of distributed systems without the benefits.

Other foundational ideas

- **Layers**: presentation (UI) → application/business logic → data access → database. Business rules shouldn't live in the UI or in SQL scattered across files.
- **APIs as contracts**: the interface between components (or with mobile apps and partners) should be explicit, documented (OpenAPI), versioned, and changed carefully. Internals can change freely behind a stable API.
- **Modularity, coupling and cohesion**: a good module does one thing (high **cohesion**) and depends on few others through narrow interfaces (low **coupling**). Then changes stay local.
- **Synchronous vs asynchronous**: request/response for things the user waits on; queues and events (Part 7) for everything else.
- **Build vs buy**: buy (or use managed services and open source) for anything that isn't your differentiator: auth, payments, email, hosting, analytics. Build what makes you different. For an AI coaching product, the scoring rubric, feedback quality and learning loop are the product; login isn't.
- **Architecture decision records (ADRs)**: a short note per significant decision: context, options considered, decision, consequences. Future teammates (and future you) learn *why*, not just *what*.

Design documents

A **design doc** (also called an RFC or tech spec) is written before significant building: context and goals, non-goals, the proposed design (diagrams, data model, APIs), alternatives considered and why rejected, risks, rollout and testing plan, open questions. Its value is that disagreements surface on paper in a day instead of in code after a month. Reading one well means asking: what problem, what's out of scope, what alternatives were considered, what happens when each dependency fails, how will we know it works, how do we roll it back?

5. Design principles and patterns

Principles are rules of thumb for keeping code changeable. Patterns are named, reusable solutions to recurring problems. You need to recognise them, not recite them.

Principles

- **Separation of concerns**: each part handles one aspect (UI, business rules, storage).
- **DRY** (don't repeat yourself): one source of truth for each piece of knowledge. Balanced by **"duplication is cheaper than the wrong abstraction"**: wait until you see a pattern three times before abstracting.
- **KISS and YAGNI**: keep it simple; you aren't gonna need it. Don't build for imagined future requirements.
- **SOLID** (object-oriented guidelines): **S**ingle responsibility (one reason to change); **O**pen/closed (extend without editing); **L**iskov substitution (subtypes must work wherever the parent does); **I**nterface segregation (small, specific interfaces); **D**ependency inversion (depend on abstractions, so you can swap a real LLM client for a fake one in tests).
- **Composition over inheritance**: build behaviour from small parts rather than deep class hierarchies.
- **Make illegal states unrepresentable**: use types, enums and constraints so invalid data can't exist (a session's `kind` can only be one of three values).

Patterns worth recognising

Pattern	Problem it solves	Everyday example
Adapter	Make an incompatible interface fit	One `LLMClient` interface with adapters for different providers, so switching models is a config change
Strategy	Swap algorithms at runtime	Different scoring rubrics for interview vs debate
Observer / pub-sub	Notify many listeners of an event	"Session completed" triggers scoring, analytics and an email
Factory	Centralise object creation	Create the right exporter (PDF, CSV) from a setting
Repository	Hide data-access details behind an interface	`SessionRepository.get_for_user(id)` instead of SQL in route handlers
Middleware / decorator	Wrap behaviour around a core action	Auth, logging, rate limiting around every API route
MVC and its descendants	Separate data, presentation and input handling	Most web frameworks
Retry, circuit breaker, queue-based load levelling	Resilience	Part 7

6. Technical debt

Shortcuts taken now that make future changes slower. Like financial debt, it can be a sound investment or a slow disaster depending on whether it's deliberate and repaid.

Ward Cunningham's metaphor, refined by Martin Fowler's quadrant: debt can be **deliberate or inadvertent**, **prudent or reckless**. "We'll hard-code the rubric to launch for the placement season, then make it configurable" is deliberate and prudent. "What's a test?" is reckless.

- **Interest** is paid every time someone works in that area: slower changes, more bugs, fear of touching code.
- **Signs**: simple changes take days; the same bug returns; only one person understands a module; deploys are scary; onboarding takes weeks.
- **Managing it**: make it visible (a debt list with estimated cost), pay it down where you're about to build (the "boy scout rule": leave code better than you found it), and reserve steady capacity (say 15–20%) rather than heroic rewrites.
- **Rewrites from scratch** are usually a mistake: the old system's ugliness often encodes hard-won fixes for edge cases. Prefer incremental replacement (the **strangler fig** pattern: route piece by piece to the new system).
- **AI-generated code adds a new kind of debt**: code nobody on the team fully understands. Review it like a colleague's, require tests, and keep architecture decisions human.

7. Estimation and managing scope and risk

Estimates are usually wrong in the same direction. Plan for that instead of being surprised by it.

Why estimates miss:

- **Unknown unknowns**: integration problems, edge cases, unclear requirements, dependencies on other teams.
- **The planning fallacy**: people estimate the best case and call it the expected case.
- **Optimistic sums**: ten tasks each with a small chance of going badly almost guarantee that some go badly; risks add up, they don't average out.
- **Brooks's law**: "adding people to a late software project makes it later", because new people need onboarding and communication paths grow roughly with the square of team size.
- **Hofstadter's law**: it always takes longer than you expect, even when you account for Hofstadter's law.

Practices that help:

- **Break work down** until each piece is a few days at most; unknown pieces get a time-boxed **spike** (investigation) first.
- **Ranges, not points**: "2–4 weeks, 80% confident". Or three-point estimates (optimistic, likely, pessimistic).
- **Relative sizing** (story points, T-shirt sizes) plus measured **velocity** or throughput is often more reliable than hours.
- **Reference-class forecasting**: how long did similar projects actually take? Usually the best predictor.
- **Fix time, flex scope**: commit to a date and a goal; negotiate what's in. The iron triangle (scope, time, cost/people, with quality underneath) means you can't fix all of them.
- **Risk register**: list risks with likelihood, impact, owner and mitigation; attack the biggest unknowns first.
- **Critical path**: the longest chain of dependent tasks sets the earliest finish; speeding anything else doesn't help.

8. Testing strategy

Tests are how a team changes code confidently. The goal isn't coverage; it's catching the failures that would hurt.

Level	Tests	Speed	Use for
Unit	One function or class in isolation	Milliseconds	Logic, edge cases, calculations
Integration	Components together (app + real database)	Seconds	Queries, API endpoints, external adapters
End-to-end	The whole app through the UI, like a user	Minutes; flaky	A handful of critical journeys (sign up, practise, see feedback, pay)
Manual / exploratory	A human trying to break it	Slow	New features, UX, things automation misses

The **testing pyramid**: many unit tests, fewer integration tests, very few end-to-end tests. Also:

- **Regression tests**: every bug fixed gets a test so it never returns.
- **Test-driven development (TDD)**: write the failing test first. Useful discipline, especially when directing AI to write code: the test defines what "correct" means.
- **Contract tests** for APIs between teams or services.
- **Load tests** before big launches (placement season traffic).
- **Evals for AI features** (Part 5): a fixed set of inputs with expected properties, run on every prompt or model change, are the test suite for LLM behaviour.
- **Code review**: a second pair of eyes catches design problems tests can't. Keep changes small so reviews are real.

9. Releasing and running software

Shipping is the beginning of the feedback loop, not the end of the project.

- **Environments**: local → preview/staging → production, with the same build promoted through them.
- **Release practices**: small, frequent releases are safer than big rare ones (smaller blast radius, easy to see what broke). Feature flags, canaries and quick rollback (Part 7) make it routine.
- **Versioning**: **semantic versioning** for APIs and libraries: MAJOR.MINOR.PATCH, where a major bump means breaking changes. Mobile apps need backwards-compatible APIs because users don't update.
- **DORA metrics** (from the DevOps Research and Assessment programme) measure delivery performance: **deployment frequency**, **lead time for changes**, **change failure rate** and **time to restore service**. High performers do well on all four at once; speed and stability are not a trade-off when practices are good.
- **Documentation that earns its keep**: a README to run the project, ADRs for decisions, API docs, runbooks for operations. Not exhaustive specs nobody reads.
- **Measure after launch**: did the feature move the metric it was meant to? Kill or rework features that don't.

10. Working with engineers as a non-engineer

Bring problems, context and priorities; let engineers own the how; argue about trade-offs in the open.

What good looks like:

- **Bring the problem and the why**, with evidence (user quotes, data), not a pre-designed solution. Engineers often find a cheaper path to the same outcome.
- **Be clear about priority and constraints**: deadline and why it's real, budget, what can be cut.
- **Ask good questions**:
- "What's the simplest version that would let us learn whether this works?"
- "What are we most unsure about, and how do we find out early?"
- "What happens when [dependency] fails or is slow?"
- "How will we know it's working in production?"
- "What would make this hard to undo?"
- "What are we trading off: what gets slower, costlier or harder later?"
- **A good trade-off discussion** names the options, the costs of each (time, money, risk, future flexibility), who bears them, and how reversible the choice is. Amazon's framing: **two-way doors** (reversible; decide fast) vs **one-way doors** (irreversible; decide carefully).
- **Respect focus**: interruptions are expensive for deep work (Paul Graham's *maker's schedule*). Batch questions; protect blocks of time.
- **Don't negotiate estimates down**; negotiate scope. Pressuring an estimate just produces a wrong one.
- **Credit and blame**: credit the team publicly; own failures as the person who set priorities.

11. Building with AI coding tools

AI writes more of the code each year. The human's job moves toward specifying, reviewing, testing and owning the design.

- **Specify precisely**: clear requirements, acceptance criteria and examples produce dramatically better generated code. A design doc and test cases are also the best prompt.
- **Keep changes small and reviewable**, just as with a human colleague.
- **Read and run** what's generated: does it handle errors, edge cases, security (Part 6), and does it match the architecture? Watch for invented APIs, silently swallowed errors, secrets in code, and over-complicated solutions.
- **Tests as the safety net**: have the AI write tests, but read them; a test that asserts the wrong thing gives false confidence.
- **Own the architecture**: let AI fill in implementations within a structure you understand, not choose the structure for you.
- **Keep a project memory**: a short file (README or a CLAUDE.md-style note) with conventions, commands and decisions, so every session starts with context.

12. Labs

Lab 1. One-page design doc. For the first version of a product you're building, write: problem and users; goals and non-goals; the main user journey; the data model (tables and key fields); the main components and how they talk (a box-and-arrow diagram); external dependencies and what happens when each fails; privacy and security notes; how you'll measure success; open questions. One page, then cut it shorter.

Lab 2. Stories and acceptance criteria. Write 8 user stories for that first version, each with Given/When/Then acceptance criteria. Mark the 3 that make the MVP.

Lab 3. Prioritise with RICE. Score 10 candidate features with RICE in a spreadsheet. Then ask whether the ranking matches your gut, and if not, which input you got wrong.

Lab 4. Milestones and risks. Break the design into 4–6 milestones, each ending in something a user could try. List the top 5 risks with mitigations, and identify which milestone retires the biggest unknown first.

Lab 5. Read a real design doc. Find a public design doc or RFC (many open-source projects publish them; Rust, Python PEPs, Kubernetes KEPs). Answer the six reading questions from Section 4.

13. Self-check questions

1. **What's the core idea behind agile?** Shorten the loop between deciding and learning, because feedback is the scarcest input; ceremonies are optional tools for that.
2. **When is waterfall reasonable?** Stable, well-understood requirements, heavy regulation, or fixed-scope contracts.
3. **What makes a good user story?** A specific user, a capability, and an outcome; small, testable, valuable alone, with acceptance criteria.
4. **Why write non-goals?** They stop scope creep and make trade-offs explicit.
5. **What's an MVP really for?** Testing the riskiest assumption with real users as cheaply as possible.
6. **Monolith or microservices for a new product?** A modular monolith; microservices solve many-team coordination problems and bring distributed-systems costs.
7. **Coupling vs cohesion?** Coupling is how much modules depend on each other (keep low); cohesion is how focused a module is (keep high).
8. **Give an example of the adapter pattern in an AI product.** One LLM client interface with provider-specific adapters, so switching models is a configuration change.
9. **When is technical debt acceptable?** When it's deliberate, understood, and there's a plan to repay it, such as hitting a launch window.
10. **Why do estimates run late?** Unknown unknowns, best-case thinking, and risks that compound rather than average out.
11. **What's Brooks's law?** Adding people to a late software project makes it later, due to onboarding and communication overhead.
12. **What does "fix time, flex scope" mean?** Commit to a date and goal, and adjust what's included rather than the date or quality.
13. **What's the testing pyramid?** Many fast unit tests, fewer integration tests, very few slow end-to-end tests.
14. **Name the four DORA metrics.** Deployment frequency, lead time for changes, change failure rate, time to restore service.
15. **One-way vs two-way doors?** Irreversible decisions deserve care; reversible ones should be made quickly.

14. Resources

Check these are still available before committing time.

- **Titus Winters, Tom Manshreck and Hyrum Wright, *Software Engineering at Google*** (free online at abseil.io). How a large organisation thinks about code review, testing, documentation and scale.
- **Frederick Brooks, *The Mythical Man-Month*.** Old, short, still right about estimation, communication and conceptual integrity.
- **Ryan Singer, *Shape Up*** (free at basecamp.com/shapeup). Shaping work, appetites, fixed time and variable scope.
- **Marty Cagan, *Inspired*.** Product management from discovery to delivery; pair with Teresa Torres's *Continuous Discovery Habits*.
- **Martin Fowler's website** (martinfowler.com) on technical debt, refactoring, microservices and the strangler fig pattern.
- **Nicole Forsgren, Jez Humble and Gene Kim, *Accelerate*.** The research behind the DORA metrics.
- **John Ousterhout, *A Philosophy of Software Design*.** Short and practical on complexity, modules and interfaces.